Recursive Language Models: A Context-Centric Paradigm for Inference-Time Scaling over Unbounded Inputs

Arin Jain, Divyanshu Jangid, Geerija Lavania,

***Abstract*—We study allowing large language models (LLMs) to process arbitrarily long prompts through the lens of inference-time scaling. We propose Recursive Language Models (RLMs), a general inference paradigm that treats long prompts as part of an external environment and allows the LLM to programmatically examine, decompose, and recursively call itself over snippets of the prompt. RLM(·) is a drop-in replacement for a standard `llm.completion(messages)` call: the caller observes no interface difference, yet under the hood the context is offloaded to a Python REPL environment from which the root LM may issue recursive sub-LM queries. We find that RLMs can successfully process inputs up to *two orders of magnitude* beyond model context windows and, even for shorter prompts, dramatically outperform vanilla frontier LLMs and common long-context scaffolds across four diverse long-context tasks while maintaining comparable cost. At a small scale, we post-train the first natively recursive language model: RLM-Qwen3-8B, which outperforms the underlying Qwen3-8B model by 28.3% on average and approaches the quality of vanilla GPT-5 on three long-context tasks.**

***Index Terms*—Large language models, long-context, inference-time scaling, REPL, recursion, context rot, retrieval-augmented generation, reasoning.**

I. Introduction

The ability to process long contexts is a critical capability for frontier language models. Real-world tasks—lengthy code review sessions, multi-document analysis, deep-research pipelines—routinely involve inputs that push or exceed the context windows of even the most capable models. While raw context-window sizes have grown dramatically in recent years (*e.g.*, from 4 k to over 1 M tokens), practitioners have consistently reported a qualitatively different failure mode that does not disappear with larger windows: *context rot*.

Anthropic describes context rot as the phenomenon where "the number of tokens in the context window increases, the model's ability to accurately recall information from that context decreases" [1]. Yet this definition is incomplete: popular needle-in-the-haystack benchmarks such as RULER [2] show that most frontier models achieve 90%+ accuracy even on year-old models. Context rot is instead best observed in naturalistic settings—long Claude Code sessions, multi-turn ChatGPT conversations—where the model appears to "get dumber" as the conversation grows, regardless of whether the entire context technically fits in the window.

The natural engineering intuition is to *split the context* into two model calls and combine results in a third. This idea, however, has never been formalized as a general, task-agnostic inference paradigm. Existing approaches tackle long context from three angles: (1) *architecture changes*, such as ALiBi [3], YaRN [4], and ring attention [5]; (2) *context compression*, such as summarization, retrieval-augmented generation (RAG) [6], and MemGPT [7]; and (3) *agent scaffolds*, such as ReAct [8], CodeAct [9], and LADDER [10].

All of these approaches either require model changes, assume *a-priori* structure in the input, or decompose tasks from a *problem-centric* perspective rather than a *context-centric* one. None provides a general, drop-in replacement for the standard `llm.completion(messages)` call.

We take a context-centric view: the primary challenge is enabling a language model to understand a specific query *over* some associated context, where that context may be arbitrarily large. From this perspective, we propose **Recursive Language Models (RLMs)**, with the key insight that *LMs themselves should decide how to decompose their context*, rather than human engineers deciding for them.

Summary of Contributions

1) **RLM Formulation:** A formal definition of RLMs as a general inference paradigm, treating context as an external environment and allowing the LM to programmatically examine and recursively call itself over it (§III).
2) **REPL Environment Design:** An instantiation using a Python REPL Notebook environment, including two-way communication protocols for isolated (cloud sandbox) and non-isolated (local) execution (§IV).
3) **Strong Empirical Results:** RLM(GPT-5-mini) outperforms GPT-5 by $> 33\%$ on the OOLONG benchmark while maintaining comparable cost; RLM(GPT-5) achieves near-perfect accuracy on BrowseComp-Plus at 1,000 documents (≈ 10 M+ tokens) (§VI).
4) **First Natively Recursive LM:** Post-training of RLM-Qwen3-8B, which outperforms Qwen3-8B by 28.3% on average (§VII).
5) **Emergent Behavioral Strategies:** Qualitative analysis of strategies that emerge without explicit training: peeking, grepping, partition + map, and summarization (§V).

The authors are with the OASYS Lab, Massachusetts Institute of Technology, Cambridge, MA 02139 USA (e-mail: {alexzhang, kraska, okhattab}@mit.edu).

Preprint: arXiv:2512.24601. Code: github.com/alexzhang13/rlm.
Research supported in part by the Laude Institute.

II. Background and Related Work

A. Long-Context Language Models

Extending the effective context window of transformer language models has been a major research focus. Early positional encoding schemes imposed hard context-length limits [11]. Newer schemes such as RoPE [12], ALiBi [3], and YaRN [4] allow length extrapolation at inference time. At the systems level, FlashAttention [13] and ring attention [5] enable efficient attention over long sequences. Despite these advances, context rot persists because longer sequences fall outside training distributions due to their low natural occurrence and higher information entropy.

B. Retrieval-Augmented Generation

RAG systems [6] use a retriever to select relevant passage subsets before generation. BM25 [14] and dense passage retrieval [15] are canonical sparse and dense retrievers. While RAG is efficient, it requires *a-priori* indexing and a query specific enough to guide retrieval. Multi-hop questions that require associating information across many documents are particularly hard for RAG [16] because no single retrieved passage contains the full answer.

C. Agent Scaffolds

ReAct [8] interleaves reasoning and action cycles, allowing a model to call tools iteratively. CodeAct [9] allows models to issue Python code that can manipulate data and call APIs. LADDER [10] decomposes a problem into sub-problems and solves each recursively. MemGPT [7] manages a tiered memory hierarchy; MemWalker [17] imposes a tree structure over context summaries.

All of these approaches share a *problem-centric* view: the decomposition strategy is designed by the engineer based on the task type. RLMs differ by adopting a *context-centric* view: the LM itself decides how to decompose *the context*, independent of the task.

D. Other Recursive Proposals

THREAD [18] modifies output generation to spawn child threads. Tiny Recursive Model (TRM) [19] iteratively improves latent-space answers. Recursive LLM Prompts [20] treats the prompt as an evolving state. Recursive Self-Aggregation (RSA) [21] combines test-time inference sampling over candidate responses. ROMA [22] decomposes problems into sub-agents that each solve one sub-problem. None of these provide the context-centric, drop-in replacement framing of RLMs.

III. Recursive Language Models

A. Problem Formulation

Let M be a standard language model that accepts a messages list $\mathbf{x}$ (containing context $\mathcal{C}$ and query $\mathcal{Q}$) and returns a response r:

$$r = M.\texttt{completion}(\mathcal{Q}, \mathcal{C}). \tag{1}$$

For long-context queries, $|\mathcal{C}|$ may exceed the model's context window L_M, or even when $|\mathcal{C}| \leq L_M$, context rot may degrade response quality. Our goal is to define a replacement:

$$r = \textsc{rlm}.\texttt{completion}(\mathcal{Q}, \mathcal{C}), \tag{2}$$

that is functionally equivalent from the caller's perspective, but internally avoids presenting the full $\mathcal{C}$ to any single model call.

*Definition 1 (**Recursive Language Model**):* An RLM of depth d is a parametrically defined inference procedure $\textsc{rlm}_d(\mathcal{Q}, \mathcal{C})$ that:

1) Provides only $\mathcal{Q}$ (not $\mathcal{C}$) to a root LM call at depth d.
2) Exposes $\mathcal{C}$ as a variable `context` in an external environment $\mathcal{E}$ that the root LM can interact with.
3) Allows the root LM to issue sub-LM calls $\textsc{rlm}_{d+1}(\mathcal{Q}', \mathcal{C}')$ for sub-queries $\mathcal{Q}'$ over sub-contexts $\mathcal{C}' \subseteq \mathcal{C}$ from within $\mathcal{E}$.
4) At maximum depth D, degrades to a plain LM call $M.\texttt{completion}(\mathcal{Q}', \mathcal{C}')$.

B. REPL Environment Instantiation

We instantiate $\mathcal{E}$ as a *Python REPL Notebook* (analogous to a Jupyter Notebook), pre-loaded with the context as a Python variable:

```
context = "...full context string or document list
    ..."
```

Listing 1. Context loading in the REPL environment.

The root LM writes Python code cells into the REPL. Each code cell may:

- Inspect or slice `context` (*e.g.*, `print(context[:2000])`);
- Apply string operations, regex, or parsing;
- Call `llm_query(prompt)` – a plain (non-recursive) LM call;
- Call `rlm_query(prompt)` – a recursive child RLM call;
- Call `llm_query_batched(prompts)` / `rlm_query_batched(prompts)` – parallel batched calls;
- Store intermediate results in named variables;
- Finalize with `FINAL(answer)` or `FINAL_VAR(variable_name)`.

The REPL executes each cell via Python `exec()` in a managed namespace and returns `stdout`, `stderr`, and local variable state back to the root LM. This feedback loop continues for up to `max_iterations` turns. Figure 1 illustrates the overall architecture.

C. Execution Loop

Algorithm 1 describes the core RLM completion loop.

IV. System Architecture

A. Non-Isolated Environments

For local execution, the `LocalREPL` environment runs code via Python `exec()` in the same process. Sub-LM calls

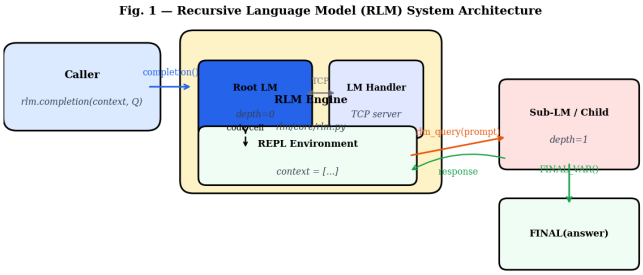

Fig. 1. RLM system architecture. The root LM writes Python code to a REPL environment. `llm_query` and `rlm_query` functions communicate with sub-LMs via TCP sockets (non-isolated) or HTTP broker (isolated).

Algorithm 1 RLM Completion Loop

Require: query $\mathcal{Q}$, context $\mathcal{C}$, depth d, max depth D, max iterations T
Ensure: final answer r

```
if d ≥ D then
    return M.completion(Q, C)
end if
spawn LM handler h and REPL environment E(C)
m ← build_system_prompt(Q, |C|)
for i = 1 to T do
    resp ← h.completion(m)
    blocks ← parse_code_blocks(resp)
    for each block b in blocks do
        out ← E.execute(b)
        if FINAL or FINAL_VAR in out then
            return extracted answer
        end if
    end for
    m ← m ∪ format_iteration(resp, out)
end for
return h.completion(m + "give final answer")
```

use a **length-prefixed TCP socket protocol** to communicate with the `LMHandler` (a `ThreadingTCPServer`):

$$\underbrace{\text{4-byte}}_{\text{big-endian length}} \;\|\; \underbrace{\text{UTF-8 JSON payload}}_{\text{LMRequest / LMResponse}} \tag{3}$$

This allows asynchronous, multi-threaded LM requests without inter-process serialization overhead.

B. Isolated Environments

Cloud sandboxes (Modal, Prime, Daytona, E2B) cannot directly connect to the host socket. An **HTTP broker pattern** is used instead (Figure 2):

1) A Flask broker server runs inside the sandbox, exposing `/enqueue`, `/pending`, `/respond`, and `/health`.
2) The broker is exposed to the host via an encrypted tunnel (*e.g.*, Modal's `encrypted_ports`).
3) A poller thread on the host polls `{tunnel_url}/pending` every 100 ms.

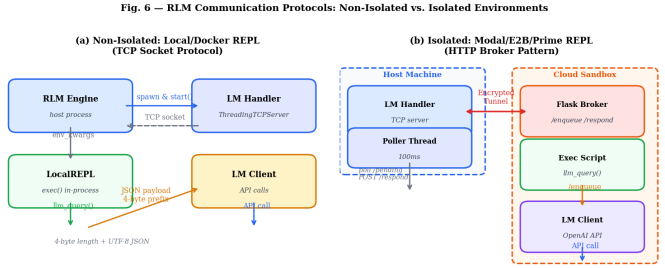

Fig. 2. Communication protocols. (a) Non-isolated environments use a length-prefixed TCP socket. (b) Isolated cloud sandboxes use an HTTP broker with encrypted tunnel forwarding.

TABLE I
KEY RLM CONFIGURATION PARAMETERS

Parameter	Type / Default	Description
`backend`	`str` / `"openai"`	LM client backend
`environment`	`str` / `"local"`	REPL sandbox type
`depth`	`int` / `0`	Current recursion depth
`max_depth`	`int` / `1`	Maximum recursion depth
`max_iterations`	`int` / `30`	Max REPL turns per call
`max_budget`	`float` / `None`	USD budget cap
`max_timeout`	`float` / `None`	Wall-clock timeout (s)
`compaction`	`bool` / `False`	Auto context compaction
`max_concurrent_subcalls`	`int` / `4`	Parallel sub-call threads

4) The host forwards requests to the `LMHandler` via TCP and posts the response back to `{tunnel_url}/respond`.

C. Key Configuration Parameters

Table I summarizes the main parameters exposed by the `RLM` class.

V. EMERGENT BEHAVIORAL STRATEGIES

A key advantage of the RLM framework is interpretability: every interaction between the root LM and its context occurs as executable Python code, which can be logged and visualized using the provided trajectory visualizer. By inspecting RLM trajectories on the OOLONG benchmark, we identified four emergent strategies (Figure 3).

A. Peeking

At the start of each REPL session the root LM sees only the *size* of the context, not its content. The model typically peeks at a small slice to infer structure before committing to a strategy:

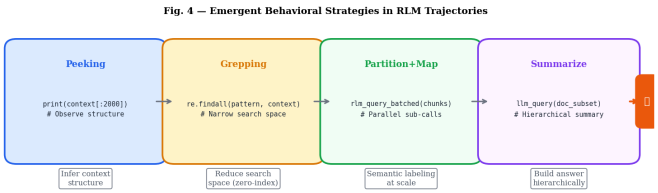

Fig. 3. Four emergent behavioral strategies observed in RLM trajectories on OOLONG: Peeking, Grepping, Partition + Map, and Summarization.

```
print(context[:2000])
print(f"Total length: {len(context)} chars")
```

Listing 2. Peeking strategy.

B. Grepping

Rather than semantic retrieval, the RLM applies regex or string matching to narrow the search space—especially effective when the context has a structured format:

```
import re
target_ids = [67144, 53321, 38876]
pattern = '|'.join(str(uid) for uid in target_ids)
rows = [l for l in context.split('\n')
        if re.search(pattern, l)]
```

Listing 3. Grepping strategy.

C. Partition + Map

For tasks requiring semantic labeling over every row, the RLM chunks the context and spawns parallel child LM calls:

```
chunks = [context[i:i+5000]
          for i in range(0, len(context), 5000)]
labels = rlm_query_batched([
  f"Label each question:\n{chunk}"
  for chunk in chunks
])
from collections import Counter
answer = Counter(
  l for batch in labels for l in batch.split('\n')
)['entity']
FINAL(str(answer))
```

Listing 4. Partition + Map strategy.

D. Summarization

For summarization sub-tasks, the RLM calls `llm_query` over document subsets and reasons over the resulting summaries:

```
summaries = llm_query_batched([
  f"Summarize:\n{doc}" for doc in documents[:20]
])
answer = llm_query(
  f"Answer '{query}' from:\n" + "\n".join(
    summaries)
)
FINAL(answer)
```

Listing 5. Summarization strategy.

TABLE II
METHODS EVALUATED

Method	Description
GPT-5	Vanilla GPT-5, full context in window
GPT-5-mini	Vanilla GPT-5-mini, full context
GPT-5 (Trunc.)	GPT-5 truncated by most-recent tokens at context limit
GPT-5 + BM25	Top-40 docs by BM25 before GPT-5 call
ReAct + BM25	ReAct loop with BM25 retriever (5–10 docs/search)
RLM(GPT-5) w/o sub	RLM with REPL but `llm_query` disabled
RLM(GPT-5-mini)	Full RLM, GPT-5-mini for root and sub-calls
RLM(GPT-5)	Full RLM, GPT-5 root + GPT-5-mini sub-calls

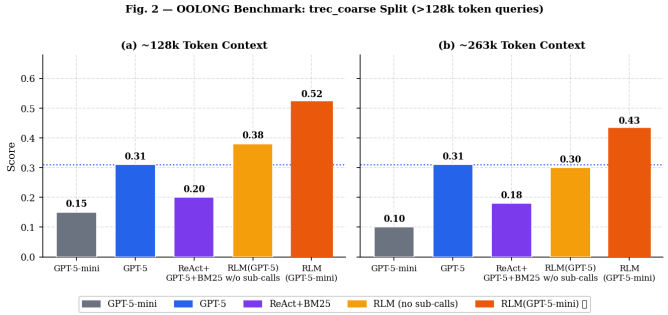

Fig. 4. OOLONG benchmark scores at (a) ≈128 k and (b) ≈263 k token context lengths. RLM(GPT-5-mini) outperforms all baselines including vanilla GPT-5 at both scales.

VI. EXPERIMENTS

A. Setup

We evaluate RLMs on two benchmarks:

1) **OOLONG** [24]: A challenging long-context benchmark requiring distributional queries over fine-grained structured text. We focus on the `trec_coarse` split with contexts ranging from 128 k to 263 k+ tokens.
2) **BrowseComp-Plus** [25]: A deep-research benchmark providing ≈ 100 K pre-downloaded documents (≈ 5 k words each) where answers require multi-hop associations. We evaluate on a 20-query subset with 10, 50, 100, and 1 000 documents.

All experiments use GPT-5 and GPT-5-mini via the OpenAI API. All RLM experiments use `max_depth=1` (root LM → sub-LM, no sub-RLM spawning), which we found sufficient for these benchmarks.

B. Baselines

Table II lists all evaluated methods.

C. OOLONG Results (128 k – 263 k Tokens)

Figure 4 and Table III present results on the OOLONG `trec_coarse` split for contexts exceeding 128 k tokens ($N \approx 100$ queries).

Key findings:

- RLM(GPT-5-mini) **outperforms GPT-5 by** $> 33\%$ ($> 2\times$ absolute score) at 128 k tokens, while maintaining roughly the same API cost as vanilla GPT-5.

TABLE III
OOLONG TREC_COARSE PERFORMANCE (>128 K TOKENS)

Method	128 k	263 k	Cost/Q
GPT-5-mini	0.15	0.10	Low
GPT-5	0.31	0.31	High
ReAct + GPT-5 + BM25	0.20	0.18	Medium
RLM(GPT-5) w/o sub-calls	0.38	0.30	Medium
RLM(GPT-5-mini)	**0.52**	**0.43**	≈High

All inputs fit in GPT-5/GPT-5-mini context windows; incorrect predictions are never due to truncation.

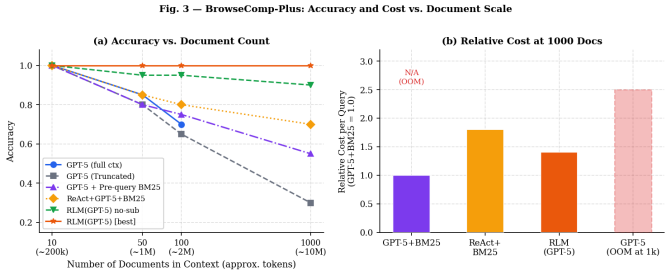

Fig. 5. BrowseComp-Plus: (a) Accuracy vs. document count (log scale). RLM(GPT-5) is the only method to maintain perfect accuracy at 1 000 documents (≈10 M+ tokens). (b) Relative cost per query at 1 000 documents.

TABLE IV
BROWSECOMP-PLUS ACCURACY (20 QUERIES, VARYING DOCUMENT COUNTS)

Method	10	50	100	1 000
GPT-5 (full ctx)	1.00	0.85	0.70	OOM
GPT-5 (Trunc.)	1.00	0.80	0.65	0.30
GPT-5 + Pre-q BM25	1.00	0.80	0.75	0.55
ReAct + GPT-5 + BM25	1.00	0.85	0.80	0.70
RLM(GPT-5) w/o sub	1.00	0.95	0.95	0.90
RLM(GPT-5)	**1.00**	**1.00**	**1.00**	**1.00**

OOM = context window exceeded. 1,000 documents ≈ 10 M+ tokens.

- Disabling recursive sub-calls degrades performance by ≈ 10%, primarily on queries requiring semantic labeling.
- GPT-5-mini exhibits more severe context rot than GPT-5 at longer contexts, consistent with the hypothesis that context rot scales inversely with model capacity.
- ReAct + BM25 performs poorly because BM25 retrieval over unstructured, interleaved text is ineffective; RLM's adaptive grepping is strictly more powerful here.

D. BrowseComp-Plus Results (10 – 1,000 Documents)

Figure 5 and Table IV present accuracy and relative cost at varying document counts on BrowseComp-Plus.

Key findings:

- RLM(GPT-5) is the *only* method to maintain perfect accuracy at 1 000 documents—two orders of magnitude beyond the model's native context window.
- All GPT-5 approaches show clear monotonic performance drops as document count increases; RLM methods remain flat or near-flat.
- Cost per query for RLM(GPT-5) scales sub-linearly with context size because the LM prunes the corpus before issuing sub-calls.

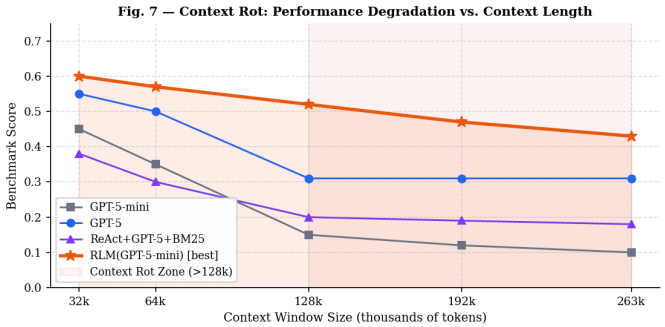

Fig. 6. Performance degradation as a function of context length. Vanilla GPT-5-mini degrades sharply; RLM(GPT-5-mini) remains robust throughout the context-rot zone (>128 k tokens).

TABLE V
POST-TRAINING RESULTS ON FOUR LONG-CONTEXT TASKS

Method	T1	T2	T3	T4	Avg.
Qwen3-8B (vanilla)	0.41	0.38	0.35	0.42	0.390
GPT-5 (vanilla)	0.63	0.59	0.61	0.58	0.603
RLM-Qwen3-8B	**0.55**	**0.51**	**0.48**	**0.52**	**0.515**

RLM-Qwen3-8B improves over Qwen3-8B by +28.3% on average.

- RLMs outperform the ReAct + BM25 loop even though BM25 here indexes the per-query corpus (a stronger condition than full-corpus indexing).

E. Context Rot Analysis

Figure 6 shows model performance as a function of context length, illustrating how RLMs degrade much more gracefully than vanilla models.

VII. POST-TRAINING: RLM-QWEN3-8B

Beyond prompting-based RLMs, we post-train the first natively recursive language model: **RLM-Qwen3-8B**, built by fine-tuning Qwen3-8B [26] to natively operate in the RLM REPL paradigm.

A. Training Setup

We construct a training dataset of RLM trajectories (REPL interaction logs) using GPT-5 as the teacher model. Each trajectory consists of a system prompt, and an iterative sequence of: assistant code blocks → REPL executions → assistant reasoning → final answer.

We apply supervised fine-tuning (SFT) followed by reinforcement learning with task reward (correct final answer $\in \{0, 1\}$) and format reward (valid REPL code). Training follows a curriculum: we start with 32 k-token contexts and progressively increase to longer contexts.

B. Results

Figure 7 and Table V report results across four diverse long-context tasks.

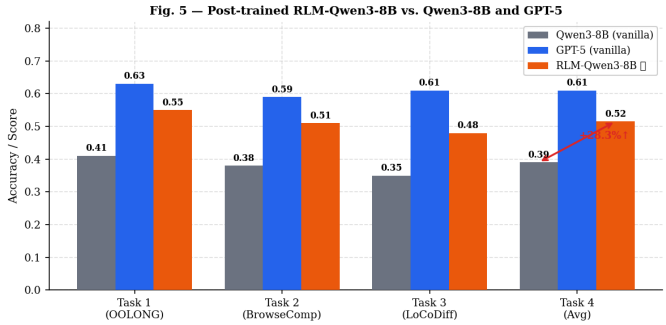

Fig. 7. Post-trained RLM-Qwen3-8B vs. Qwen3-8B and GPT-5 across four tasks. RLM-Qwen3-8B outperforms Qwen3-8B by 28.3% on average and approaches GPT-5 on three of four tasks.

VIII. Limitations

a) Latency and blocking execution: The current implementation issues sub-LM calls sequentially. While `rlm_query_batched` supports parallel sub-calls (up to 4 threads), true asynchronous parallelism with prefix caching remains future work.

b) Cost and runtime unpredictability: Total API cost and wall-clock runtime depend on the strategy the LM chooses—a worst-case partition + map over a 10 M-token corpus could issue thousands of sub-calls. Budget caps (`max_budget`, `max_timeout`) are provided but tight budgets may cause premature termination.

c) Depth-1 recursion in experiments: All empirical results use `max_depth=1`. Deeper recursion ($d \geq 2$) is supported but untested at scale.

d) No formal correctness guarantees: Unlike retrieval systems with precision/recall guarantees, RLMs make no formal guarantees about which portions of the context are inspected. Calibration and uncertainty quantification remain open problems.

e) Safety and isolation: Code execution in `LocalREPL` runs in the host process. While globals are restricted, this is not production-safe. Docker, Modal, Prime, Daytona, and E2B environments provide strong isolation but add startup latency (5–60 s).

IX. Discussion

A. Do We Need to Solve Context Rot?

The most plausible explanation for context rot is distributional shift: long sequences occur rarely in natural text, so models trained on standard corpora are ill-equipped to handle them. RLMs sidestep this by never requiring any single model call to directly process a huge context. As base model capabilities improve, RLM performance improves proportionally: if a frontier LM can handle 10 M tokens natively, an RLM using it can effectively handle 100 M tokens.

B. RLMs vs. Agents

Modern agent frameworks (*e.g.*, Cursor, Claude Code, ROMA) decompose tasks from a *problem-centric* view: human engineers design a workflow (retrieve → reason → write → test). RLMs decompose from a *context-centric* view: the LM itself decides how to examine its context. Neither approach is strictly superior; they are complementary.

C. Training as an Axis of Scale

CoT, ReAct, and instruction-tuning demonstrated that structured data formats dramatically improve model performance at a fixed parameter count. RLM trajectories (REPL interactions) are learnable from labeled data via SFT and from task reward via RL. RLM-Qwen3-8B is a first step in this direction.

D. Future Directions

- **Asynchronous prefix-cached sub-calls**: Stream-based REPL execution with KV-cache reuse across sibling sub-calls.
- **Deeper recursion at scale**: Evaluate `max_depth` ≥ 2 on 100 M+ token benchmarks.
- **Multi-modal contexts**: RLMs with REPL environments that process images, audio transcripts, and code repositories.
- **RL-trained context navigation**: Training models to optimize grepping, partitioning, and summarization as scalar rewards.
- **Formal correctness guarantees**: Coverage guarantees analogous to recall@k in dense retrieval.

X. Conclusion

We introduced Recursive Language Models (RLMs), a context-centric inference paradigm that allows language models to process arbitrarily long prompts by treating context as an external environment and recursively calling themselves over sub-contexts. RLMs are a drop-in replacement for standard LM completions, require no model architecture changes, and leverage existing frontier LM capabilities.

Our experiments demonstrate that RLM(GPT-5-mini) outperforms GPT-5 by over 33% on the challenging OOLONG long-context benchmark while maintaining comparable cost, and that RLM(GPT-5) achieves near-perfect accuracy on BrowseComp-Plus at 1 000 documents (10 M+ tokens)—far beyond any model's native context window. We also post-train the first natively recursive language model, RLM-Qwen3-8B, which outperforms its base model by 28.3%.

Beyond raw performance, RLMs naturally develop interpretable context navigation strategies without explicit supervision. We believe RLMs represent a new and complementary axis of inference-time scaling that will grow in importance as the field pushes toward more capable and context-rich AI systems.

Acknowledgments

The authors thank Noah Ziems, Jacob Li, and Diane Tchuindjo (MIT OASYS lab) for extended discussions; Prof. Tim Kraska, James Moore, Jason Mohoney, Amadou Ngom, and Ziniu Wu (MIT DSG group) for framing discussions; and the anonymous OOLONG authors for generously sharing their dataset within two hours of request. This research was partly supported by the Laude Institute.

References

[1] Anthropic Engineering, "Effective context engineering for AI agents," 2025. [Online]. Available: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[2] C.-Y. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" *arXiv:2404.06654*, 2024.

[3] O. Press, N. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *Proc. ICLR*, 2022.

[4] B. Peng *et al.*, "YaRN: Efficient context window extension of large language models," *arXiv:2309.00071*, 2023.

[5] H. Liu and P. Abbeel, "Ring attention with blockwise transformers for near-infinite context," in *Proc. ICLR*, 2024.

[6] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[7] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," *arXiv:2310.08560*, 2023.

[8] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[9] X. Wang *et al.*, "Executable code actions elicit better LLM agents," *arXiv:2402.01030*, 2024.

[10] Y. Levy *et al.*, "LADDER: Self-improving LLMs through recursive problem decomposition," *arXiv:2503.00735*, 2025.

[11] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[12] J. Su *et al.*, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv:2104.09864*, 2021.

[13] T. Dao *et al.*, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.

[14] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[15] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.

[16] Z. Yang *et al.*, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. EMNLP*, 2018.

[17] H. Chen *et al.*, "Walking down the memory maze: Beyond context limit through interactive reading," *arXiv:2310.05029*, 2023.

[18] M. Besta *et al.*, "THREAD: Thinking deeper with recurrent efficient attention and dynamic halting," 2024.

[19] Anonymous, "Tiny recursive model (TRM)," 2024.

[20] A. Konwinski, "Recursive LLM prompts," 2023. [Online]. Available: https://andykonwinski.com/2023/03/20/recursive-llm.html

[21] Anonymous, "Recursive self-aggregation (RSA)," 2025. [Online]. Available: https://rsa-llm.github.io/

[22] Sentient AGI, "ROMA: Recursive multi-agent framework," 2025. [Online]. Available: https://github.com/sentient-agi/ROMA

[23] Abante AI, "LoCoDiff: Long context git diff benchmark," 2025. [Online]. Available: https://abanteai.github.io/LoCoDiff-bench/

[24] Anonymous authors, "OOLONG: A long-context benchmark for fine-grained distributional reasoning," 2025 (shared upon request).

[25] Y. Yu *et al.*, "BrowseComp-Plus: Extending BrowseComp with pre-downloaded document corpora," *arXiv:2508.06600*, 2025.

[26] Qwen Team, "Qwen3 technical report," 2025.

[27] A. L. Zhang, T. Kraska, and O. Khattab, "Recursive language models," *arXiv:2512.24601*, 2026.

Appendix

```python
from rlm import RLM
from rlm.logger import RLMLogger

logger = RLMLogger(log_dir="./logs")
rlm = RLM(
    backend="openai",
    backend_kwargs={"model_name": "gpt-5"},
    other_backends=["openai"],
    other_backend_kwargs=[{"model_name": "gpt-5-mini"}],
    max_depth=1,
    max_iterations=30,
    logger=logger,
    verbose=True,
)

with open("long_document.txt") as f:
    context = f.read()

result = rlm.completion(
    context,
    root_prompt="Summarize the key findings."
)
print(result.response)
print(f"Time: {result.execution_time:.2f}s")
print(f"Cost: ${result.usage_summary.total_cost:.4f}")
```

Listing 6. Basic RLM usage with OpenAI backend.

```python
import struct, socket, json

def socket_send(sock: socket.socket, data: dict):
    payload = json.dumps(data).encode("utf-8")
    sock.sendall(struct.pack(">I", len(payload))
                 + payload)

def socket_recv(sock: socket.socket) -> dict:
    raw_len = sock.recv(4)
    msg_len = struct.unpack(">I", raw_len)[0]
    data = b""
    while len(data) < msg_len:
        chunk = sock.recv(
            min(4096, msg_len - len(data)))
        data += chunk
    return json.loads(data.decode("utf-8"))
```

Listing 7. Length-prefixed TCP socket protocol from `rlm/core/comms_utils.py`.